

# React Interview

Savollar va qisqa, ogʻzaki interview uchun tayyor javoblar

Qanday foydalanish: Javoblarni soʻzma-soʻz yodlashdan koʻra maʼnosini tushunib, oʻz soʻzing bilan aytishga harakat qil. Kod yozish savollarida ham tushuntirishni mashq qil.

## 1. React asoslari

### □ React nima?

Javob: React — JavaScript asosida qurilgan kutubxona boʻlib, user interface yaratish uchun ishlatiladi. UI componentlar yordamida quriladi.

### □ React nima uchun ishlatiladi?

Javob: Interaktiv va qayta ishlatiladigan UI yaratish, state boshqarish va katta frontendlarni componentlarga boʻlib qurish uchun.

### □ React library yoki framework?

Javob: React odatda JavaScript library deb ataladi. Routing yoki global state kabi koʻp imkoniyatlar alohida kutubxonalar bilan qoʻshiladi.

### □ SPA nima?

Javob: Single Page Application. Sahifa toʻliq reload boʻlmasdan, kerakli UI qismlari yangilanadigan web application.

### □ Virtual DOM nima?

Javob: React UI ning virtual koʻrinishini xotirada saqlaydi. State yoki props oʻzgarganda React oʻzgarishlarni hisoblab, kerakli DOM qismlarini yangilaydi.

### □ Component nima?

Javob: UI ning qayta ishlatiladigan kichik qismi. Component odatda katta harf bilan boshlanadi va kerak boʻlsa export/import qilinadi.

### □ Functional component nima?

Javob: UI qaytaradigan JavaScript function. Hozirgi Reactʼda functional componentlar va Hooks eng koʻp ishlatiladi.

### □ JSX nima?

Javob: JSX — JavaScript XML. JavaScript ichida HTMLʼga oʻxshash sintaksis bilan UI yozishga imkon beradi. JSX HTML emas.

### □ JSX va HTML farqi?

Javob: JSXʼda className ishlatiladi, JavaScript expressionlar {} ichida yoziladi, style object boʻlishi mumkin va ayrim atributlar camelCase yoziladi.

## 2. Props va State

### □ Props nima?

Javob: Props — parent componentdan child componentga qiymat yoki ma'lumot uzatish uchun ishlatiladigan maxsus obyekt. Props read-only.

### □ Propsni child o'zgartira oladimi?

Javob: Bevosita o'zgartirmasligi kerak. Child propsni o'qiydi. Agar child parentdagi ma'lumotni o'zgartirmoqchi bo'lsa, parent functionni prop qilib yuborishi mumkin.

### □ State nima?

Javob: Componentning ichki, o'zgaruvchan holati. State o'zgarganda React componentni qayta render qiladi.

### □ Props va State farqi?

Javob: Props parentdan keladi va child uchun read-only. State componentning o'zida boshqariladi va setter orqali yangilanadi.

### □ useState nima?

Javob: State saqlash va yangilash uchun React Hook. Masalan: `const [count, setCount] = useState(0)`.

### □ setState nima qiladi?

Javob: State uchun berilgan setter yangi qiymatni o'rnatadi va kerak bo'lsa re-renderni ishga tushiradi.

### □ Previous state qachon kerak?

Javob: Yangi state oldingi statega bog'liq bo'lsa: `setCount(prev => prev + 1)`. Bu bir nechta yangilanishlarda xavfsizroq.

### □ State object/array yangilanganda?

Javob: Immutability saqlanadi: eski object/arrayni to'g'ridan-to'g'ri mutate qilmay, yangi nusxa yaratiladi.

## 3. Hooks

### □ Hook nima?

Javob: React funksiyalariga state, effect, context va boshqa React imkoniyatlarini qo'shadigan maxsus functionlar. Hook nomi odatda use bilan boshlanadi.

### □ Hooks qoidalari?

Javob: Hooklarni component yoki custom Hookning top-level qismida chaqirish kerak; loop, condition yoki oddiy nested function ichida chaqirmaslik kerak.

### □ useEffect nima?

Javob: Side effectlar uchun Hook: API request, timer, event listener, document title kabi ishlar.

### □ Dependency array nima?

Javob: Effect qachon qayta ishlashini belgilaydi. `[]` — mountdan keyin bir marta; `[count]` — count o'zgarganda; array umuman bo'lmasa — har renderdan keyin.

### □ useEffect cleanup nima?

Javob: Effect yaratgan timer yoki listener kabi resurslarni tozalash uchun return function ishlatiladi.

### □ useRef nima?

Javob: Renderga ta'sir qilmaydigan qiymatni renderlar orasida saqlaydi va DOM elementga murojaat qilishda ishlatiladi. `ref.current` o'zgarsa re-render bo'lmaydi.

### □ **useState va useRef farqi?**

Javob: State o'zgarsa re-render bo'ladi; `ref.current` o'zgarsa re-render bo'lmaydi.

### □ **useMemo nima?**

Javob: Qimmat hisoblangan qiymatni dependencylar o'zgarmaguncha memoize qilish uchun.

### □ **useCallback nima?**

Javob: Function reference'ni dependencylar o'zgarmaguncha saqlash uchun. Ko'pincha memoized child bilan ishlaganda foydali.

### □ **React.memo nima?**

Javob: Component proplari o'zgarmaganida keraksiz qayta renderlarni kamaytirishga yordam beradigan memoization.

### □ **useContext nima?**

Javob: Contextdagi qiymatni Provider orqali chuqur componentlarga propsni qo'lda uzatmasdan olish uchun.

### □ **Custom Hook nima?**

Javob: Qayta ishlatiladigan stateful logicni function ichiga ajratish. Nomi odatda use bilan boshlanadi.

## 4. Events va Forms

### □ **onClick nima?**

Javob: Element bosilganda function ishga tushirish uchun React event handler.

### □ **onChange nima?**

Javob: Input qiymati o'zgarganda handlarni ishga tushiradi.

### □ **onSubmit nima?**

Javob: Form yuborilganda ishlaydi. Ko'pincha event.preventDefault() bilan browser reloadining oldi olinadi.

### □ **Controlled component nima?**

Javob: Input qiymati React state orqali boshqariladigan component.

### □ **Uncontrolled component nima?**

Javob: Input qiymati DOM tomonidan boshqariladi; kerak bo'lsa useRef orqali olinadi.

### □ **React Hook Form nima?**

Javob: Formlarni kamroq boilerplate bilan boshqarish, validation, errors va submit jarayonini qulaylashtirish uchun kutubxona.

### □ **register nima?**

Javob: React Hook Form'da inputni form state va validationga ulash uchun.

### □ **Controller nima?**

Javob: MUI kabi controlled componentlarni React Hook Form bilan ulashda ishlatiladi.

### □ **watch nima?**

Javob: Formdagi qiymatlarni kuzatish va UI yoki logicda ishlatish uchun.

## 5. Rendering va Lists

### □ **Conditional rendering nima?**

Javob: Shartga qarab turli UI chiqarish. Masalan: condition ? : yoki condition && .

### □ **map nima uchun?**

Javob: Array elementlaridan bir nechta React element yaratish uchun.

### □ **key nima?**

Javob: Listdagi elementni React uchun barqaror identifikatsiya qiladi va reconciliation jarayonida qaysi element o'zgarganini aniqlashga yordam beradi.

### □ **Indexni key qilish mumkinmi?**

Javob: Mumkin, lekin list tartibi o'zgarsa yoki elementlar qo'shilib/o'chsa muammolar keltirishi mumkin. Barqaror unique id afzal.

### □ **Re-render nima?**

Javob: Component functionining yana bajarilib, Reactning yangi UI natijasini hisoblash jarayoni.

### □ **React qachon re-render qiladi?**

Javob: State yoki props o'zgarganda, parent qayta render bo'lganda va ayrim context o'zgarishlarida.

## 6. Componentlar orasidagi aloqa

### □ **Parent → Child qanday data yuboradi?**

Javob: Props orqali.

### □ **Child → Parent qanday ma'lumot yuboradi?**

Javob: Parent functionni prop qilib childga beradi; child shu functionni chaqiradi.

### □ **Lifting state up nima?**

Javob: Bir nechta sibling componentga kerak bo'lgan state'ni ularning umumiy parentiga ko'tarish.

### □ **Prop drilling nima?**

Javob: Kerakli componentga yetkazish uchun propsni ko'p oraliq componentlardan ketma-ket uzatish.

### □ **children prop nima?**

Javob: Componentning ochilish va yopilish teglari orasidagi JSXni qabul qiladigan maxsus prop.

## 7. Routing

### □ **React Router nima?**

Javob: React application ichida URL va UI o'rtasidagi navigatsiyani boshqarish uchun routing kutubxonasi.

### □ **Link nima?**

Javob: Full page reloadsiz boshqa routega o'tish uchun ishlatiladi.

### □ **useNavigate nima?**

Javob: JavaScript orqali programmatic navigation qilish uchun Hook.

### □ **useParams nima?**

Javob: Dynamic URL parametrlarini olish uchun. Masalan /users/:id.

### □ **Nested route nima?**

Javob: Bir route ichida boshqa route'lar joylashishi. Parent layout ichida child route ko'rsatiladi.

### □ **Outlet nima?**

Javob: Nested route child componentlari render bo'ladigan joy.

## 8. API va Async

### □ **API nima?**

Javob: Frontend va backend o'rtasida ma'lumot almashish uchun interface.

### □ **GET, POST, PUT/PATCH, DELETE?**

Javob: GET — olish; POST — yaratish; PUT/PATCH — yangilash; DELETE — o'chirish.

### □ **fetch nima?**

Javob: HTTP request yuborish uchun browserdagi API.

### □ **async/await nima?**

Javob: Promise asosidagi asynchronous kodni o'qilishi osonroq ko'rinishda yozish usuli.

### □ **Loading va Error state nima uchun?**

Javob: API javobini kutish paytida loading UI, xatolik bo'lsa error UI ko'rsatish uchun.

### □ **API requestni qayerda qilish mumkin?**

Javob: Ko'pincha useEffect ichida, event handlerda yoki data-fetching kutubxonalari orqali.

## 9. Context, Zustand va State Management

### □ **Context API nima?**

Javob: Globalga yaqin qiymatlarni componentlar daraxti bo'ylab Provider orqali uzatish usuli.

### □ **Zustand nima?**

Javob: React uchun yengil global state management kutubxonasi. Store ichida state va actions saqlanadi.

### □ **Local va global state farqi?**

Javob: Local state bitta yoki yaqin componentlarga tegishli; global state ko'p, uzoq componentlar tomonidan ishlatilishi mumkin.

### □ **Zustand persist nima?**

Javob: Store state'ini masalan localStorage'da saqlab, sahifa reload bo'lganda qayta tiklashga imkon beradi.

### □ **Context va Zustand farqi?**

Javob: Context qiymat uzatish mexanizmi; Zustand esa maxsus state-management store va selector/action imkoniyatlariga ega.

## 10. Performance va Advanced

### □ Memoization nima?

Javob: Oldindan hisoblangan qiymat yoki function reference'ni qayta ishlatib, keraksiz hisoblash/renderlarni kamaytirish texnikasi.

### □ useMemo va useCallback farqi?

Javob: useMemo qiymatni memoize qiladi; useCallback function reference'ni memoize qiladi.

### □ Qachon memoization ishlatmaslik kerak?

Javob: Har bir joyda emas. Oddiy hisob-kitob yoki arzon component uchun ortiqcha complexity va memory sarfi berishi mumkin.

### □ Immutability nima?

Javob: Mavjud object/arrayni bevosita o'zgartirmasdan yangi nusxa yaratish prinsipi.

### □ Reconciliation nima?

Javob: React yangi va oldingi render natijalarini taqqoslab, real DOMga kerakli minimal o'zgarishlarni qo'llash jarayoni.

### □ Mount, update, unmount?

Javob: Mount — component birinchi paydo bo'lishi; update — qayta yangilanishi; unmount — DOMdan olib tashlanishi.

### □ StrictMode nima?

Javob: Developmentda muammoli patternlarni aniqlashga yordam beradigan React development tool. Ba'zi lifecycle/effectlar developmentda qo'shimcha tekshirilishi mumkin.

## 11. Interviewda ko'p tushadigan 'tricky' savollar

### □ Nega component nomi katta harf bilan boshlanadi?

Javob: React lowercase nomlarni odatda HTML element sifatida, uppercase nomlarni esa custom component sifatida talqin qiladi.

### □ Nega state'ni to'g'ridan-to'g'ri o'zgartirmaymiz?

Javob: React yangilanishini to'g'ri aniqlashi va predictability saqlanishi uchun. Setter yoki yangi object/array ishlatiladi.

### □ useEffect API uchun majburiymi?

Javob: Yo'q. API requestni event handler yoki maxsus data-fetching vositalarida ham qilish mumkin. useEffect — side effect lifecycle bilan bog'liq bo'lsa foydali.

### □ useRef o'zgarganda nega UI yangilanmaydi?

Javob: ref.current o'zgarishi React state update emas, shuning uchun re-render trigger qilmaydi.

### □ State asynchronousmi?

Javob: Setterni chaqirgandan keyin yangi state qiymati shu zahoti o'sha function scope ichida o'zgargan deb hisoblash kerak emas. React updatesni batch qilishi mumkin.

### □ React'da event oddiy DOM eventdan farq qiladimi?

Javob: React event system unified interface beradi; zamonaviy React'da eventlar native eventlar bilan ishlaydi, lekin handler yozilishi React sintaksisida bo'ladi.



### □ **default export va named export?**

Javob: default export bitta asosiy exportni beradi va import nomini o'zgartirish mumkin; named exportlar {} bilan import qilinadi.

### □ **Key sifatida random qiymat ishlatish mumkinmi?**

Javob: Tavsiya etilmaydi, chunki har renderda key o'zgarib, React elementlarni yangi deb qabul qilishi mumkin.